Arquitectura de LA TROUPE

Plataforma para academia de danza híbrida

Documento maestro de arquitectura — versión 3.2

Configurabilidad total, identidad federada y SaaS multi-tenant

24 tablas modeladas · 6 fases de roadmap · Stack productivo definido

Fecha: abril de 2026

Índice de contenidos

1. Resumen ejecutivo — 3
2. Principios core de la arquitectura — 4
3. Modelo de datos completo — 6
4. Dominio: Identidad y permisos — 7
5. Dominio: Contenido on-demand — 11
6. Dominio: Operaciones en vivo — 13
7. Dominio: Autorización (vault) — 16
8. Dominio: Monetización y facturación — 17
9. Dominio: Staff docente y pagos — 20
10. Dominio: Notificaciones — 21
11. Dominio: Auditoría y reportería — 23
12. Dominio: Configuración y plataforma — 25
13. Casos de uso críticos resueltos — 28
14. Stack tecnológico recomendado — 32
15. Roadmap de 6 fases — 35
16. Decisiones tomadas vs delegadas al panel admin — 38
17. Glosario — 40

1. Resumen ejecutivo

LA TROUPE es una plataforma para una academia de danza con modelo de negocio 100% híbrido: el alumno puede consumir contenido on-demand (pre-grabado), asistir a clases presenciales en sede física, y/o conectarse a clases virtuales en vivo vía Zoom. La arquitectura debe soportar las tres modalidades sin duplicar entidades ni reescribir lógica.

Este documento es el resultado de un análisis crítico realizado sobre la propuesta inicial de arquitectura. Consolida 18 decisiones arquitectónicas tomadas, identifica las decisiones que aún dependen de la academia, y deja un modelo de datos consistente listo para que un equipo de desarrollo lo implemente.

Lo que cambió respecto al documento original

- Se separó conceptualmente enrollments de reservations: la inscripción semestral ocupa el cupo durante toda la temporada, y la asistencia real se registra en una tabla attendance independiente.
- Se modeló una tabla waitlist con FIFO y oferta de 48h para liberación de cupos.
- Se introdujo el modelo Guardian-Student: una cuenta de usuario puede administrar múltiples perfiles de alumno (caso de padres con hijos menores).
- Se desacopló el video de la lección con una tabla lesson_assets provider-agnostic, lo que permite migrar de Bunny.net a S3 (o cualquier otro) sin tocar el modelo de negocio.
- Se preparó la arquitectura para multi-tenant desde el día uno (organization_id en todas las tablas relevantes + Row-Level Security en PostgreSQL), aunque operativamente solo exista LA TROUPE en MVP.
- Se modeló el pago a profesores con reglas de compensación flexibles (per_class, per_student, fixed_monthly, hybrid).
- Se diferenció el canal del pago (online vs POS) del método (cash, card, transferencia, redes de cobranza), y se diseñó smart payment routing para enrutar a cada alumno al gateway óptimo según su país (MercadoPago en Latam, Stripe en internacional).
- Se agregó una tabla event_log central para trazabilidad y reportes de retención.
- Se diseñó el módulo de notificaciones como servicio centralizado con templates, preferences, queue y log de envíos.

Lo que cambió respecto a la v2.1

La versión 3.0 incorpora una filosofía nueva: business rules as data. Las decisiones comerciales que en v2.1 estaban hardcodeadas en el modelo (políticas de menores, descuentos familiares, política de reembolsos, etc.) ahora viven en una tabla de configuración editable desde el panel admin. Esto permite que el owner adapte el sistema sin requerir cambios de código.

- Nueva tabla organization_settings: motor de configuración por organización con todos los settings comerciales editables desde el panel admin.
- Nueva tabla platform_subscriptions: para cuando se vendan licencias del software a otras academias (modelo SaaS).
- Nuevo campo users.is_platform_admin: distingue al equipo de la plataforma (tu empresa) de los owners de cada academia.
- Modificación crítica en courses.allowed_modes: ahora es un array que declara qué modalidades son posibles. La modalidad concreta de cada lección vive en class_sessions.location_id.
- Nuevo principio core: identidad federada con membresías múltiples. Una cuenta de usuario puede tener relación con múltiples academias del sistema.
- Stack tecnológico productivo definido: Vercel + Supabase + Bunny.net + MercadoPago + Stripe + Resend + Sentry, con costo estimado de USD 5-10/mes en MVP.

- Reorganización de la sección de decisiones: la antigua sección 15.2 de 'decisiones pendientes' se transforma en 'decisiones delegadas al panel admin', reflejando que el sistema soporta cualquier política comercial vía configuración.

Lo que cambió respecto a la v3.1

La versión 3.2 incorpora una corrección filosófica importante: detectamos que aplicamos YAGNI (You Aren't Gonna Need It) en lugares donde no debíamos. Concretamente, postergamos la captura de datos a fases tardías cuando esos datos son irrecuperables si no se capturan desde el inicio. La regla nueva: lo que no se captura hoy, no se inventa mañana.

- Nuevo principio core 2.10: 'Captura de datos desde el día uno'. Establece la diferencia entre captura de datos (debe ser temprana) y UI que consume esos datos (puede ser tardía).
- Nueva tabla staff_profile (minimal): se crea desde Fase 1 para tener histórico de RRHH desde el principio (fecha de contratación, notas internas, status). La UI completa de gestión de personal vive en Fase 5.
- Nueva tabla lesson_progress: trackea posición y completitud por alumno-lección desde Fase 1. La UI de 'continuá donde quedaste' se construye en Fase 4, pero los datos ya están desde el día uno.
- Nueva tabla lesson_video_views: captura eventos de visualización (play, pause, seek, complete) desde Fase 1. Permite analítica avanzada en Fase 4 sin perder data histórica.
- Nueva tabla lesson_chapters: capítulos como tabla aparte (no JSON dentro de lessons). Permite editar individualmente, agregar metadata, y queryear por capítulo.
- Sección 14 (Stack): los costos en USD se mantienen pero con disclaimer claro de que son referenciales para escenarios de contratación de devs externos. Se agrega columna de complejidad relativa como métrica complementaria.
- Sección de roadmap aclarada: distinción explícita entre tablas de captura (siempre Fase 1) y features visibles (puede ser fase posterior).
- Decisión sobre reproductor de video: Vidstack como base, con Speed/Capítulos/Mirror en Fase 1, Loop A-B en Fase 2, Cámara local en Fase 4, Cambiar Vistas en Fase 5 (decisión comercial primero).

2. Principios core de la arquitectura

Estos principios son la columna vertebral del sistema. Cada decisión técnica debe alinearse con ellos. Si una feature contradice un principio, se rechaza la feature, no se modifica el principio.

2.1 Agnosticismo físico/digital en tres niveles

LA TROUPE es un solo sistema inteligente que distingue lo físico de lo virtual en tres capas exactas, no en una sola:

- A nivel arquitectura base no se distingue: no hay tablas separadas para clases presenciales y online. Ambas viven en class_sessions. Online es tratada como una sede más en la tabla locations.
- A nivel seguridad y permisos sí se distingue: la tabla entitlements es la que define quién puede reservar cupo en qué sede. Un alumno virtual queda bloqueado por entitlements si intenta reservar un lugar físico.
- A nivel interfaz y UX sí se distingue: la tabla locations tiene un campo type (physical / virtual) que el frontend usa para renderizar mapa y dirección, o botón de Zoom, según corresponda.

2.2 Separación de responsabilidades: access vs entitlement

El derecho a consumir contenido digital (access) y el derecho a ocupar un lugar físico o virtual en vivo (entitlements) son dos conceptos distintos, modelados en dos tablas distintas. Esta separación es lo que permite vender membresías híbridas sin duplicar cursos.

2.3 Contenido atómico y global

Las lecciones (lessons) son la unidad mínima de contenido. Se graban y suben una sola vez y nunca se duplican. Los cursos (courses) no son dueños del contenido: son agrupadores lógicos que referencian lecciones mediante un orden específico, lo que permite curar y mezclar contenido sin replicar videos.

2.4 Desacoplamiento financiero

La forma, el canal y la ubicación donde un alumno paga (payments) son independientes de cómo y dónde consume el servicio. Las facturas (invoices) representan obligación de pago; los pagos (payments) representan ingreso de dinero. Una factura puede pagarse total o parcialmente, en uno o varios pagos, por canales distintos.

2.5 Identidad de cuenta vs identidad de alumno

Una cuenta de login (users) y un perfil de alumno (student_profiles) son entidades distintas. Una cuenta puede administrar múltiples perfiles. Esto resuelve el caso de padres que pagan por dos hijos menores con un solo email, y también soporta sin esfuerzo el caso del adulto auto-gestionado.

2.6 Multi-tenant by design, single-tenant by deployment

Todas las tablas relevantes incluyen organization_id desde el día uno, aunque en MVP solo exista LA TROUPE. PostgreSQL Row-Level Security garantiza el aislamiento de datos a nivel base de datos, no solo de aplicación. Esto permite agregar academias futuras sin refactorización.

2.7 Provider-agnostic en infraestructura crítica

Los proveedores externos (Bunny.net para video, MercadoPago para pagos, Resend para email) son detalles de implementación, no parte del modelo de negocio. Las tablas guardan referencias abstractas (provider, provider_asset_id, gateway, gateway_transaction_id) que permiten migrar de proveedor sin reescribir lógica.

2.8 Business rules as data, not as code

Las decisiones comerciales son cambiantes en el tiempo: política de reembolsos, edad mínima para cuenta propia, descuentos familiares, política con profesores que toman clases. Modelar estas reglas en código las congela; modelarlas como datos las hace adaptables al mercado.

El sistema implementa esto vía la tabla organization_settings, que actúa como motor central de configuración. Cada setting tiene una clave, un valor (JSON para flexibilidad), una categoría y un flag de editabilidad. El panel admin lee y escribe esta tabla, lo que permite al owner adaptar el comportamiento del sistema sin requerir despliegues de código.

Adicionalmente, decisiones más estructuradas (precios, descuentos, capacidades) viven directamente en sus entidades correspondientes (plans, courses, class_sessions) y son editables vía CRUD del panel admin.

2.9 Identidad federada con membresías múltiples

Cuando el sistema crezca para soportar múltiples academias (modelo SaaS), un mismo usuario puede tener relación con varias organizaciones simultáneamente. Por ejemplo: un profesor que enseña en LA TROUPE y también en Academia Z, o una alumna que toma clases en dos academias distintas.

El sistema resuelve esto con una sola cuenta de usuario global (users.email único en toda la plataforma) y múltiples membresías (memberships) que la vinculan a distintas organizaciones con distintos roles. El usuario hace login una sola vez y luego selecciona en qué organización quiere operar (similar a Slack o GitHub Organizations).

El aislamiento de datos entre organizaciones se garantiza a nivel de queries (Row-Level Security filtrando por organization_id), no a nivel de identidad. Cada academia ve solo sus propios datos; la identidad global solo comparte información mínima de login (email) entre membresías.

2.10 Captura de datos desde el día uno: lo que no se captura hoy no se inventa mañana

Este principio surge de una corrección de criterio importante. En desarrollo de software hay una regla clásica llamada YAGNI (You Aren't Gonna Need It) que dice: 'no construyas lo que no necesitás hoy'. Es buena regla para features visibles, pero peligrosa para captura de datos.

La distinción crítica es la siguiente: una feature visible (un botón, una pantalla, un reporte) se puede construir en cualquier momento del proyecto. Pero los datos que esa feature necesita consumir deben estar capturados desde antes. Si el alumno usó la plataforma 6 meses sin que trackeáramos su progreso, no podemos inventar retroactivamente dónde quedó en cada lección.

La regla operativa: si una decisión arquitectónica involucra captura de datos, NO se aplica YAGNI. Las tablas de captura (lesson_progress, lesson_video_views, attendance, event_log) se crean en Fase 1 aunque sus features visibles vengan después. El costo de capturar datos desde el día uno es bajo. El costo de no capturarlos es perder información histórica irrecuperable.

Aplicación práctica del principio

- Tablas de captura activadas desde Fase 1 aunque la UI llegue después: lesson_progress, lesson_video_views, attendance, event_log, staff_profile.
- Se permite YAGNI cuando la decisión es agregar una nueva tabla sin tocar las existentes y sin necesidad de migrar datos (ejemplo: tabla de gamificación, recompensas, achievements).
- Se permite YAGNI cuando la decisión es construir una UI nueva sobre datos que ya tenemos.
- NO se permite YAGNI cuando la decisión es 'no capturar este evento ahora porque no lo usamos todavía'. Si el evento ocurre y tiene valor potencial, se captura.

3. Modelo de datos completo

El modelo de datos final consta de 27 tablas distribuidas en 8 dominios funcionales. La tabla siguiente resume la estructura. Cada dominio se desarrolla en detalle en las secciones 4 a 12.

Dominio	Tablas	Función
Identidad y permisos	users, organizations, locations, memberships, user_auth_providers, student_profiles, teacher_profile, staff_profile	Quiénes son los usuarios, qué pueden hacer, dónde lo hacen
Contenido on-demand	lessons, courses, course_lessons, lesson_assets, lesson_chapters, lesson_progress, lesson_video_views	Material educativo digital + captura de progreso desde día uno
Operaciones en vivo	class_sessions, masterclasses, class_enrollments, attendance, waitlist, class_session_teachers	Eventos en tiempo y espacio determinado
Autorización (vault)	access, entitlements	Derechos de consumo y de asistencia
Monetización	plans, subscriptions, subscription_students, invoices, payments, purchases	Motor comercial y financiero
Staff docente	teacher_compensation_rules, teacher_payouts	Reglas y liquidaciones de pago a profesores

Comunicación y auditoría	notification_templates, notification_preferences, notifications_sent, notification_queue, event_log	Notificaciones, observabilidad, retención
Configuración y plataforma	organization_settings, platform_subscriptions	Reglas comerciales editables y SaaS multi-tenant

Convenciones del modelo

- Toda tabla incluye id (UUID), created_at, updated_at.
- Toda tabla relevante incluye organization_id (excepto users y platform_subscriptions, que son globales por diseño).
- Las relaciones uno-a-muchos se modelan con foreign keys explícitas.
- Las relaciones muchos-a-muchos se modelan con tablas pivote, no con campos JSON o arrays.
- Los campos enumerados (status, type, etc.) se modelan con CHECK constraints o tablas de catálogo, no con strings libres.
- Las fechas críticas (expires_at, due_date, canceled_at) son nullable explícitamente para distinguir 'no aplica' de 'pendiente'.
- Los settings comerciales (políticas de la academia) viven en organization_settings, no en columnas hardcodeadas.
- Las tablas de captura de datos (lesson_progress, lesson_video_views, attendance, event_log) se activan desde Fase 1 aunque su UI consumidora llegue en fases posteriores.

4. Dominio: Identidad y permisos

4.1 Tabla users

Entidad base universal del sistema. Representa una cuenta de login global, no necesariamente un alumno. Un usuario puede ser un padre que administra alumnos, un profesor, un administrador, un alumno auto-gestionado, o el equipo de la plataforma. La distinción se hace vía memberships, student_profiles e is_platform_admin.

Campo	Tipo	Notas
id	UUID	Primary key
email	string	Único globalmente, normalizado a lowercase + trim
email_verified_at	timestamp	Null si no verificado
password_hash	string	Nullable: si solo usa OAuth, no tiene
name	string	Nombre
lastname	string	Apellido
phone	string	Opcional, formato internacional
is_platform_admin	boolean	True solo para el equipo de la plataforma (acceso al Platform Admin Panel)
last_active_organization_id	UUID	Última organización seleccionada (para UX del switch)
created_at	timestamp	Auditoría
updated_at	timestamp	Auditoría

Crítico: el email es único en TODA la plataforma (no por organización). Esto permite identidad federada: un mismo usuario puede tener relación con múltiples academias vía memberships. La separación de datos entre academias se logra a nivel de queries (Row-Level Security), no a nivel de identidad.

4.2 Tabla user_auth_providers

Permite a un usuario tener múltiples métodos de login conectados a la misma cuenta (account linking). Resuelve el caso de un alumno que se registra con email y luego conecta Google, o viceversa.

Campo	Tipo	Notas
id	UUID	Primary key
user_id	UUID	FK a users
provider	enum	password, google, apple, facebook
provider_user_id	string	ID externo del proveedor (null para password)
connected_at	timestamp	Cuándo se conectó este método

Auto-link silencioso: si un usuario se registra con un proveedor cuyo email coincide con una cuenta existente, el sistema solicita verificación del método existente y, una vez verificado, agrega un nuevo registro en esta tabla. Nunca se permite vincular sin verificación previa.

4.3 Tabla organizations

Soporte multi-tenant. En MVP existe una sola organización (LA TROUPE), pero todas las queries del sistema deben filtrar por organization_id. PostgreSQL Row-Level Security está activo desde el día uno como red de seguridad arquitectónica.

Campo	Tipo	Notas
id	UUID	Primary key
name	string	Nombre comercial
slug	string	Identificador URL-friendly
timezone	string	Para cálculos de horarios
currency	string	Moneda principal de operación
created_at	timestamp	Auditoría

4.4 Tabla locations

Sedes de la organización. El campo type es metadata para la UX (mostrar mapa vs link de Zoom) y para reportes. No define permisos de acceso: eso lo resuelve la tabla entitlements.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
name	string	Ej: 'Sede Montevideo' o 'Online'
type	enum	physical / virtual
address	string	Solo si type=physical
zoom_account_id	string	Solo si type=virtual
default_capacity	integer	Cupo por defecto al crear sesiones
timezone	string	Override de la organización si difiere

4.5 Tabla memberships

Núcleo del modelo de roles. Define el vínculo entre un usuario y una organización con tres dimensiones de permisos.

Campo	Tipo	Notas
id	UUID	Primary key
user_id	UUID	FK a users
organization_id	UUID	FK a organizations
system_role	enum	owner, admin, manager, staff (configuración)
domain_role	enum	teacher, student (función académica)
staff_role	enum	cashier, receptionist (RBAC operativo)
location_id	UUID	Opcional: staff atado a sede física
status	enum	active, suspended, terminated

Nota sobre staff: en el alcance actual, los roles de staff (cashier, receptionist, manager, admin) no requieren una tabla de 'perfil' adicional porque su identidad está completamente definida por users + memberships. Si en el futuro el sistema necesita guardar datos específicos de RRHH (salario base, fecha de contratación, comisiones de venta, metas de cobranza, vacaciones), se puede crear una tabla staff_profile siguiendo el mismo patrón de teacher_profile. No se implementa ahora por YAGNI (You Aren't Gonna Need It), pero el modelo lo soporta sin refactorización.

4.6 Tabla student_profiles (modelo Guardian-Student)

Esta es una de las decisiones arquitectónicas más importantes. Separa la identidad de cuenta (users) de la identidad de alumno (student_profiles). Un usuario puede administrar múltiples perfiles de alumno, lo que resuelve naturalmente el caso de padres con hijos menores y el caso del adulto auto-gestionado.

Importante sobre cuándo se crea: el student_profile NO se crea automáticamente al registrarse en la plataforma. Se crea cuando la persona declara intención de ser alumna, ya sea inscribiéndose a su primer curso o explícitamente desde 'Mi cuenta → Quiero tomar clases'. Esto evita que la base de datos se llene de perfiles vacíos de gente que se registró pero nunca tomó una clase (padres administradores que solo gestionan a sus hijos, profesores que nunca son alumnos, admins contratados, etc.). El reporte de 'alumnos activos' debe contar registros reales, no cuentas creadas.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
account_user_id	UUID	Quién administra este perfil
first_name	string	Nombre del alumno real
last_name	string	Apellido del alumno real
birth_date	date	Para validar edad y aplicar tarifas
guardian_relationship	enum	self, parent, guardian, other
mode	enum	presencial, virtual, híbrido
medical_notes	text	Alergias, lesiones, condiciones físicas
photo_consent	boolean	Autorización para fotos en redes
status	enum	active, paused, archived

Implicancia crítica: todas las tablas que se refieren al alumno (enrollments, attendance, access, entitlements, waitlist) usan student_profile_id, no user_id. Las tablas que se refieren a quien paga (invoices, payments, subscriptions, purchases) usan user_id (el account_user).

4.7 Tabla teacher_profile

Datos específicos del rol de profesor. Un mismo user puede tener teacher_profile y student_profile simultáneamente sin problemas.

Campo	Tipo	Notas
id	UUID	Primary key
user_id	UUID	FK a users
organization_id	UUID	FK a organizations
bio	text	Biografía pública
specialties	string[]	Estilos en los que enseña
is_external	boolean	True si es internacional/contratado para masterclass
status	enum	active, on_leave, terminated

4.8 Jerarquía completa de niveles

El sistema tiene cinco niveles jerárquicos. Es importante distinguir el Platform Owner (tu empresa, dueño del SaaS) del Organization Owner (dueño de una academia individual).

Nivel	Quién es	Cantidad	Modelado vía
Platform Owner / Admin	Tu empresa: dueño del software entero	1 entidad (varios usuarios)	users.is_platform_admin = true
Organization Owner	Dueño de una academia (ej: dueño de LA TROUPE)	1 por organización	memberships.system_role = owner
Organization Admin	Configura su academia, no puede borrarla	Varios por organización	memberships.system_role = admin
Manager / Staff / Teacher	Roles operativos dentro de la academia	Muchos por organización	memberships.system_role + staff_role + domain_role
Student	Usuario final consumidor	Muchos por organización	memberships.domain_role = student

Diferencia crítica: Platform Owner vs Organization Owner

- Platform Owner (tu empresa): puede crear/suspender organizaciones, ver métricas globales del SaaS, acceder a un panel cross-tenant, pero NO ve datos de los alumnos de cada academia (eso lo bloquea Row-Level Security a nivel arquitectónico).
- Organization Owner (dueño de academia): control total dentro de su academia, configura settings, gestiona usuarios, ve todos los datos de su organización, pero NO ve datos de otras academias del sistema.
- El Platform Owner factura a las academias el uso del software (vía platform_subscriptions). Cada Organization Owner factura a sus alumnos el servicio educativo (vía invoices).

Identidad federada: el caso del usuario en múltiples academias

Cuando una academia adicional entra al sistema (ej: Academia Z), un usuario existente puede vincular su cuenta a la nueva academia sin crear cuenta duplicada.

- Flujo: María, alumna de LA TROUPE, intenta registrarse en Academia Z con su mismo email.
- El sistema detecta que el email ya existe globalmente y solicita autenticación con el método existente.
- Una vez verificada, se crea un membership nuevo con organization_id = Academia Z.

- María hace login una sola vez y luego usa un selector de organización (UI tipo Slack workspace switcher) para cambiar entre LA TROUPE y Academia Z.
- Sus student_profiles, enrollments y datos académicos están separados por organización: Academia Z nunca ve los datos de LA TROUPE.
- Lo único que se comparte entre academias es la identidad básica de login (email, nombre).

Poderes excepcionales del Platform Owner

El Platform Owner es el único nivel del sistema que puede ver el grafo completo de memberships de un usuario a través de todas las academias. Este poder está reservado para situaciones específicas y limitadas, no es de uso cotidiano.

- Soporte técnico: cuando un usuario reporta un problema cross-academy ('mi cuenta de LA TROUPE no me deja entrar a Academia Z'), el Platform Owner puede diagnosticar el problema sin pedir credenciales.
- Resolución de disputas: si dos academias del sistema reclaman que un usuario les está abusando, el Platform Owner puede investigar.
- Migración de datos: cuando una academia se da de baja del SaaS, el Platform Owner gestiona la transferencia o el archivado de datos.
- Auditoría legal: si una autoridad solicita información sobre un usuario específico, el Platform Owner es quien responde.

Salvaguardas críticas: el Platform Owner ve la estructura (qué memberships tiene un usuario, en qué academias está) pero NO ve los datos sensibles operativos (asistencia, facturas detalladas, contenido consumido). Cada acceso a información cross-tenant queda registrado en event_log con el motivo. Esta es una decisión de diseño no técnica: legalmente, las academias confían sus datos a la plataforma asumiendo este nivel de privilegios mínimos del Platform Owner.

4.9 Email único como decisión arquitectónica consciente

El campo users.email es único en TODA la plataforma, no por organización. Esto no es una coincidencia técnica: es una decisión consciente con implicancias importantes que conviene explicitar para que ningún desarrollador futuro las cuestione sin entender por qué.

Qué significa exactamente

- Si María se registra en LA TROUPE con maria@gmail.com, ningún otro usuario en ninguna academia futura puede tomar ese email.
- Si Academia Z entra al sistema y otra María (sin relación con la primera) intenta registrarse con maria@gmail.com, el sistema rechaza el intento y le indica que ese email ya tiene cuenta global.
- La 'segunda María' debe usar otro email para registrarse en Academia Z, o (si es la misma persona) vincular su cuenta existente.
- Esta restricción es a nivel base de datos (UNIQUE constraint), no solo a nivel aplicación. Es arquitectónicamente imposible saltarla.

Por qué este modelo es correcto

- Permite identidad federada: el mismo usuario puede usar la misma cuenta en múltiples academias del sistema.
- Reduce fricción: los usuarios no recuerdan 20 contraseñas distintas.
- Facilita el modelo SaaS: cuando crece la plataforma, los usuarios fluyen entre academias sin re-registrarse.
- Estándar de la industria: Slack, Notion, Linear, GitHub Organizations operan así.

Consecuencias y trade-offs

- Trade-off 1: en caso de homonimia (dos personas distintas que casualmente tienen emails similares), una de las dos debe usar otro email. En la práctica esto es muy raro porque los emails son únicos por su naturaleza.

- Trade-off 2: si una academia futura quisiera 'aislamiento total' donde sus usuarios sean completamente invisibles para otras academias, este modelo no se lo permite. La identidad de email es compartida, aunque los datos operativos están aislados.
- Trade-off 3: el Platform Owner sabe en qué academias está cada usuario (por consultar memberships), aunque no sepa qué hace en cada una. Las academias deben confiar en esta arquitectura.

Si en el futuro alguna academia exige aislamiento total de identidad, eso requeriría un modelo diferente (sub-tenancy con namespaces de email). En ese caso, sería una academia de tipo 'enterprise tier' con arquitectura aparte. Pero eso está fuera del alcance del modelo SaaS estándar de LA TROUPE.

4.10 Tabla staff_profile (minimal)

Tabla incorporada en v3.2 siguiendo el principio 2.10 (captura de datos desde el día uno). Guarda metadata específica de los usuarios que cumplen roles de staff (cashier, receptionist, manager, admin). En Fase 1 los campos son mínimos; el módulo completo de RRHH llega en Fase 5, pero el dato histórico empieza a acumularse desde el primer día.

Campo	Tipo	Notas
id	UUID	Primary key
user_id	UUID	FK a users
organization_id	UUID	FK a organizations
hired_at	date	Fecha de contratación o vinculación
terminated_at	date	Fecha de finalización (null si sigue activo)
internal_notes	text	Notas internas de RRHH (privadas, solo admin)
status	enum	active, on_leave, terminated

Importante: esta tabla se crea en Fase 1 vacía o con datos mínimos. La UI completa de gestión de personal (asignación de comisiones, vacaciones, evaluaciones) se construye en Fase 5. Pero el dato de 'cuándo se contrató' y 'cuándo se promovió' empieza a registrarse desde el día uno. Si en Fase 5 querés calcular antigüedad de un cobrador, el sistema sabrá responder.

5. Dominio: Contenido on-demand

Material educativo digital, global y atómico. Las lecciones (lessons) son la unidad mínima y existen una sola vez. Los cursos (courses) son agrupadores lógicos que referencian lecciones mediante orden específico. Esto permite curar y mezclar contenido sin duplicar videos.

5.1 Tabla lessons

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	Contenido privado por organización
title	string	Título de la lección
description	text	Descripción detallada
style_id	UUID	FK a tabla de catálogo styles
level	enum	beginner, intermediate, advanced
duration_seconds	integer	Duración total (suma de assets video)

is_free	boolean	True si es lección gratuita completa
status	enum	draft, published, archived

5.2 Tabla courses

Importante: el campo allowed_modes declara qué modalidades son posibles para este curso, pero la modalidad concreta de cada lección/sesión vive en class_sessions.location_id (cuyo location.type es physical o virtual). Esto permite que un curso híbrido tenga, por ejemplo, lecciones teóricas virtuales y prácticas presenciales, decidido sesión por sesión.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	Contenido privado por organización
name	string	Ej: 'Urbano Completo'
description	text	Descripción del producto
allowed_modes	string[]	Array: ['presencial'], ['virtual'], o ['presencial', 'virtual']
enrollment_mode	enum	semester, weekly, drop_in
start_date	date	Solo si enrollment_mode=semester
end_date	date	Solo si enrollment_mode=semester
cover_image_url	string	Imagen de marketing
status	enum	draft, published, archived

Ejemplo concreto: el curso 'Calle PRO Híbrido' tiene allowed_modes = ['presencial', 'virtual']. Cuando se programan sus class_sessions, el admin decide para cada una si es presencial (location_id apuntando a sede Montevideo) o virtual (location_id apuntando a sede Online). Las lecciones teóricas pueden ser virtuales y las prácticas presenciales, o viceversa, sin restricciones del sistema.

5.3 Tabla course_lessons

Tabla pivote que define qué lecciones forman parte de un curso y en qué orden.

Campo	Tipo	Notas
course_id	UUID	FK a courses
lesson_id	UUID	FK a lessons
order	integer	Posición en el curso
is_locked_until	integer	Días desde inicio para desbloquear (drip)

5.4 Tabla lesson_assets

Provider-agnostic. Permite que una misma lección tenga video principal, thumbnail, subtítulos en varios idiomas, y previews cortos. Si en el futuro se migra de Bunny.net a S3, se agregan registros nuevos con provider distinto sin tocar el resto del sistema.

Campo	Tipo	Notas
id	UUID	Primary key
lesson_id	UUID	FK a lessons

asset_type	enum	video, thumbnail, subtitle, preview
provider	enum	bunny, vimeo, s3, youtube
provider_asset_id	string	ID del recurso en el proveedor
quality	enum	480p, 720p, 1080p, master
duration_seconds	integer	Solo para video
language	string	ISO 639-1, solo para subtitle
is_preview	boolean	True si es preview gratuito corto
created_at	timestamp	Auditoría

Estrategia de contenido gratuito

LA TROUPE tendrá dos tipos de contenido gratuito complementarios:

- Lecciones gratuitas completas (lessons.is_free = true): la lección entera es de acceso libre. Útil para 'lección 1 introductoria' de cada curso.
- Previews cortos por asset (lesson_assets.is_preview = true): los primeros 3 minutos de un video específico. Útil para mostrar muestra antes del paywall.

Validación de acceso a una lección

La lógica de autorización debe vivir en una sola función centralizada (canUserAccessLesson). Pseudocódigo:

- 1. Si lesson.is_free = true → SÍ.
- 2. Si el usuario tiene un registro en access para el curso de esa lección, no expirado → SÍ.
- 3. En cualquier otro caso → NO (mostrar paywall).

5.5 Tabla lesson_chapters

Capítulos o secciones dentro de una lección. Permite que el alumno navegue rápido a 'calentamiento', 'coreo parte 1', 'repaso final', etc. La barra de progreso del reproductor muestra marcadores en estos timestamps.

Esta tabla se modela aparte (no como JSON dentro de lessons) porque permite editar capítulos individualmente, queryear por capítulo, agregar metadata específica (thumbnail, dificultad), y soportar features futuras como 'marcar capítulo favorito' o 'progreso por capítulo'.

Campo	Tipo	Notas
id	UUID	Primary key
lesson_id	UUID	FK a lessons
order	integer	Orden visual del capítulo dentro de la lección
title	string	Ej: 'Calentamiento', 'Coreo Parte 1'
description	text	Opcional, descripción del capítulo
start_time_seconds	integer	Timestamp de inicio dentro del video
end_time_seconds	integer	Timestamp de fin (nullable: hasta el siguiente capítulo)
thumbnail_url	string	Opcional, preview visual del capítulo
difficulty	enum	beginner, intermediate, advanced (opcional)

5.6 Tabla lesson_progress

Trackea el progreso de cada alumno en cada lección. Tabla incorporada desde Fase 1 siguiendo el principio 2.10 (captura de datos desde el día uno). En Fase 4 se construye la UI de 'continuá donde quedaste', pero los datos ya están

desde el primer alumno que entra a la plataforma.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
student_profile_id	UUID	FK a student_profiles
lesson_id	UUID	FK a lessons
current_position_seconds	integer	Última posición vista del video
total_watched_seconds	integer	Suma acumulada de segundos vistos (incluye repeticiones)
completion_percentage	decimal	0-100, calculado en función del total visto y la duración
status	enum	not_started, in_progress, completed
first_watched_at	timestamp	Primera vez que abrió la lección
last_watched_at	timestamp	Última vez que la abrió
completed_at	timestamp	Cuándo cruzó el umbral de completado

Criterio de 'completado': cuando total_watched_seconds supera el 85% de la duración de la lección. Se usa 85% en lugar de 100% porque los alumnos típicamente saltan los créditos finales. La regla puede configurarse en organization_settings si la academia quiere ajustarla.

5.7 Tabla lesson_video_views

Captura eventos detallados de visualización: qué partes del video se ven, cuántas veces se repiten, a qué velocidad. Tabla incorporada desde Fase 1 siguiendo el principio 2.10. En Fase 4 alimenta el dashboard analítico del Owner ('qué partes de las lecciones repiten más los alumnos'), pero los eventos se capturan desde el día uno.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
student_profile_id	UUID	FK a student_profiles
lesson_id	UUID	FK a lessons
event_type	enum	play, pause, seek, complete, mirror_toggle, loop_set
segment_start	integer	Segundo de inicio del segmento (para play/pause)
segment_end	integer	Segundo de fin del segmento (nullable)
playback_speed	decimal	Velocidad usada en el momento del evento
mirror_active	boolean	Si el modo espejo estaba activo
device_type	enum	desktop, tablet, mobile
occurred_at	timestamp	Cuándo ocurrió el evento

Esta tabla puede crecer mucho en volumen. En MVP es manejable. Cuando llegue a millones de eventos por mes, se puede archivar a un sistema analítico separado (Snowflake, BigQuery) sin perder los datos. La estrategia se planifica en Fase 6.

6. Dominio: Operaciones en vivo

Todo lo que ocurre en un tiempo y espacio determinado. Cada tabla de este dominio lleva organization_id y location_id (cuando aplica). El dominio resuelve el corazón operativo de la academia: inscripciones, cupos, asistencia y lista de espera.

6.1 Tabla class_sessions

Un evento específico: cruza un curso con una sede, una fecha-hora y una capacidad.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
course_id	UUID	FK a courses
location_id	UUID	FK a locations
scheduled_at	timestamp	Fecha y hora de inicio
duration_minutes	integer	Duración planificada
capacity	integer	Cupo máximo (puede diferir por sede)
actual_attendance_count	integer	Denormalizado para reportes rápidos
status	enum	scheduled, in_progress, completed, cancelled
zoom_meeting_url	string	Solo si location.type=virtual

6.2 Tabla masterclasses

Eventos premium de pago único. Generalmente dictados por profesores internacionales contratados exclusivamente para ese evento.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
title	string	Título
description	text	Descripción de marketing
teacher_id	UUID	FK a teacher_profile
location_id	UUID	FK a locations
scheduled_at	timestamp	Fecha y hora
duration_minutes	integer	Duración
price	decimal	Precio único
capacity	integer	Cupo máximo
available_until	timestamp	Hasta cuándo se pueden vender entradas

6.3 Tabla class_enrollments

La inscripción semestral del alumno a un curso. Ocupa el cupo durante toda la duración del enrollment. No se generan reservas semanales: cada class_session del curso queda implícitamente cubierta.

Campo	Tipo	Notas
id	UUID	Primary key

organization_id	UUID	FK a organizations
student_profile_id	UUID	FK a student_profiles
course_id	UUID	FK a courses
location_id	UUID	Sede en la que ocupa cupo
enrolled_at	timestamp	Cuándo se inscribió
start_date	date	Inicio del semestre
end_date	date	Fin del semestre
status	enum	active, dropped, completed
dropped_at	timestamp	Si se dio de baja, cuándo
completed_at	timestamp	Si completó, cuándo

6.4 Tabla attendance

Fuente de verdad para 'qué clases tomó realmente el alumno'. Crítica para reportes de horas de carrera de danza y para cálculo de pagos a profesores con esquema per_student.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
enrollment_id	UUID	FK a class_enrollments
class_session_id	UUID	FK a class_sessions
status	enum	present, absent, justified_absence, late
recorded_by	UUID	FK a users (profesor o staff)
recorded_at	timestamp	Cuándo se pasó lista
notes	text	Justificación, certificado, observaciones

6.5 Tabla waitlist

Lista de espera FIFO por curso. Cuando un alumno se da de baja, el sistema notifica al primero de la lista con una oferta de 48h. Si no confirma, pasa al siguiente.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
student_profile_id	UUID	FK a student_profiles
course_id	UUID	FK a courses
location_id	UUID	Sede deseada
position	integer	Orden FIFO
status	enum	waiting, offered, accepted, expired, declined
offered_at	timestamp	Cuándo se le ofreció el cupo
offer_expires_at	timestamp	Hasta cuándo tiene para aceptar
created_at	timestamp	Cuándo se anotó

6.6 Tabla class_session_teachers

Tabla pivote que asocia profesores a sesiones. Permite múltiples profesores por sesión, sustituciones registradas con auditoría completa, y co-dictado. La presencia real del profesor en la clase se registra acá, no en class_sessions.

Campo	Tipo	Notas
id	UUID	Primary key
class_session_id	UUID	FK a class_sessions
teacher_id	UUID	FK a teacher_profile
role	enum	titular, substitute, co_titular, assistant
status	enum	scheduled, confirmed, replaced, no_show, taught
compensation_override	decimal	Override puntual de la regla general
created_at	timestamp	Auditoría

7. Dominio: Autorización (vault)

El corazón del sistema híbrido. Dos tablas que parecen similares pero resuelven problemas distintos. La separación es lo que permite vender membresías híbridas sin duplicar cursos.

7.1 Tabla access — derecho de consumo digital

Define si un alumno puede darle 'play' a los videos de un curso. No tiene location_id porque el contenido digital no tiene fronteras.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
student_profile_id	UUID	FK a student_profiles
course_id	UUID	FK a courses (nullable si es por masterclass)
masterclass_id	UUID	FK a masterclasses (nullable si es por curso)
granted_by	enum	purchase, subscription, gift, trial
source_id	UUID	ID del purchase/subscription que lo otorgó
granted_at	timestamp	Cuándo se otorgó
expires_at	timestamp	Cuándo caduca (null = perpetuo)

7.2 Tabla entitlements — derecho de asistencia

Define en qué sedes (físicas o virtuales) el alumno tiene permiso para ocupar un cupo. Cuando el motor de reservas (class_enrollments) actúa, consulta entitlements para validar que el alumno puede inscribirse en esa sede específica.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
student_profile_id	UUID	FK a student_profiles
course_id	UUID	FK a courses

location_id	UUID	FK a locations (null = todas las sedes)
granted_by	enum	purchase, subscription, manual
source_id	UUID	ID del purchase/subscription que lo otorgó
granted_at	timestamp	Cuándo se otorgó
expires_at	timestamp	Cuándo caduca (null = perpetuo)

Caso de uso resuelto: Calle PRO híbrido

Curso 'Calle PRO' con dos tipos de alumnos:

- Alumno VIP: tiene access para ver videos + entitlement con location_id = NULL (puede asistir en cualquier sede física o virtual).
- Alumno Económico: tiene access para ver videos + entitlement con location_id = Online (solo puede reservar cupo virtual).
- Si el alumno Económico intenta inscribirse a la sede Montevideo, el motor de reservas lo bloquea porque no tiene entitlement para esa location_id.

8. Dominio: Monetización y facturación

8.1 Tabla plans

Ofertas comerciales del sistema. Cada plan define su tolerancia financiera (grace_period_days), su política familiar (max_students), y su tipo de ciclo (rolling vs calendar).

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
name	string	Ej: 'Plan Híbrido Familiar'
description	text	Descripción comercial
price	decimal	Precio mensual
billing_cycle	enum	monthly, quarterly, yearly
billing_calendar	enum	rolling, fixed_month_start
type	enum	on_demand, presencial, hibrido, family
max_students	integer	1=individual, N=familiar
grace_period_days	integer	3 para online, 15-30 para presencial
is_proratable	boolean	True si calcula prorrateo a inicio de mes
status	enum	active, archived

8.2 Tabla subscriptions

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
user_id	UUID	Quien paga (account_user)

plan_id	UUID	FK a plans
status	enum	active, grace_period, suspended, canceled
started_at	timestamp	Inicio del ciclo
current_period_start	date	Inicio del período facturado
current_period_end	date	Fin del período facturado
canceled_at	timestamp	Cuándo se canceló
canceled_by	UUID	FK a users (quién la canceló)
cancellation_reason	string	Motivo de baja
cancellation_category	enum	financial, conduct, relocation, other

8.3 Tabla subscription_students

Vincula una suscripción con los alumnos cubiertos. Resuelve los planes familiares: una suscripción 'Plan Familiar 2 Hijos' tiene dos registros en esta tabla.

Campo	Tipo	Notas
id	UUID	Primary key
subscription_id	UUID	FK a subscriptions
student_profile_id	UUID	FK a student_profiles
added_at	timestamp	Cuándo se agregó
removed_at	timestamp	Cuándo se removió (null=activo)

8.4 Tabla invoices

Obligación de pago. Generada automáticamente por el sistema según el ciclo de la suscripción, o manualmente por el staff para pagos puntuales.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
user_id	UUID	A quién se factura
subscription_id	UUID	FK a subscriptions (nullable)
amount_total	decimal	Monto total
amount_paid	decimal	Monto efectivamente pagado
currency	string	Moneda
issued_at	timestamp	Fecha de emisión
due_date	date	Vencimiento
paid_at	timestamp	Cuándo se saldó completamente
status	enum	open, paid, partial, void, uncollectible
is_prorated	boolean	True si fue prorrateada

8.5 Tabla payments

Ingreso real de dinero. Una factura puede tener uno o varios pagos asociados.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
user_id	UUID	Quien pagó
invoice_id	UUID	FK a invoices
amount	decimal	Monto del pago
currency	string	Moneda
channel	enum	online_self_service, pos_in_store
method	enum	cash, card_present, card_online, bank_transfer, mercadopago, dlocal, redes_cobranza
gateway	string	Proveedor (mercadopago, dlocal, abitab, redpagos)
gateway_transaction_id	string	ID externo del proveedor
location_id	UUID	Solo si channel=pos_in_store
processed_by	UUID	Solo si channel=pos_in_store (cajero)
status	enum	pending, succeeded, failed, refunded
processed_at	timestamp	Cuándo se procesó

8.6 Tabla purchases

Ticket de compra. Conecta lo que se compró con el pago que lo originó.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
user_id	UUID	Quien compró
course_id	UUID	Si compró un curso (nullable)
plan_id	UUID	Si compró un plan (nullable)
masterclass_id	UUID	Si compró una masterclass (nullable)
payment_id	UUID	FK a payments
amount	decimal	Monto pagado por este item
created_at	timestamp	Auditoría

Lógica de prorrateo (planes presenciales/híbridos)

Para optimizar la operativa de caja en sede, los planes presenciales usan calendario fijo:

- Vencimiento unificado: todos los alumnos presenciales vencen el día 1 del mes.
- Si el alumno se inscribe el día 15, el sistema genera una primera factura prorrateada por los días restantes del mes (16 días).
- La siguiente factura se programa al día 1 del mes siguiente, alineando al ciclo fijo.
- Para planes 100% digitales se usa rolling: cada 30 días desde la compra.

Ciclo de vida de la suscripción

Estado	Cuándo entra	Qué pasa con accesos
active	Facturas al día	access y entitlements activos

grace_period	Factura vencida pero dentro del grace	access y entitlements siguen activos
suspended	Grace agotado	access y entitlements REVOCADOS
canceled	Cancelación explícita	access y entitlements terminados

9. Dominio: Staff docente y pagos

Este dominio se activa en Fase 5 del roadmap, pero los datos crudos se capturan desde Fase 3 para no tener agujeros históricos al momento de calcular los primeros pagos.

9.1 Tabla teacher_compensation_rules

Reglas de pago a profesores. Permite tres modelos puros (per_class, per_student, fixed_monthly) y un modelo híbrido (base + variable). Una regla puede aplicar globalmente o solo a un curso específico.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
teacher_id	UUID	FK a teacher_profile
course_id	UUID	Nullable: aplica solo a un curso
compensation_type	enum	per_class, per_student, fixed_monthly, hybrid
base_amount	decimal	Monto base (per_class o componente fijo)
per_student_amount	decimal	Monto por alumno asistente
currency	string	Moneda
valid_from	date	Vigencia desde
valid_until	date	Vigencia hasta (null = vigente)

9.2 Tabla teacher_payouts

Liquidación periódica (típicamente mensual). El sistema calcula automáticamente el monto sumando attendance + reglas vigentes.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
teacher_id	UUID	FK a teacher_profile
period_start	date	Inicio del período liquidado
period_end	date	Fin del período liquidado
total_classes	integer	Cantidad de clases dictadas
total_students_taught	integer	Suma de alumnos asistentes
amount_total	decimal	Monto a pagar
status	enum	draft, approved, paid, voided
paid_at	timestamp	Cuándo se pagó al profesor

10. Dominio: Notificaciones

Módulo centralizado para envío de notificaciones por múltiples canales: email, WhatsApp, SMS y in-app. Diseñado como servicio interno con cola, retry automático y log de envíos.

10.1 Tabla notification_templates

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
event_type	enum	class_reminder, payment_due, etc.
channel	enum	email, whatsapp, sms, in_app
language	string	ISO 639-1
subject_template	string	Solo email
body_template	text	Con variables {{name}} {{date}}
is_active	boolean	Para A/B testing y deprecación

10.2 Tabla notification_preferences

Preferencias por usuario × tipo de evento × canal. Crítica para cumplimiento legal (GDPR/equivalentes): el usuario puede opt-out de marketing.

Campo	Tipo	Notas
user_id	UUID	FK a users
event_type	enum	Tipo de evento
channel	enum	Canal específico
enabled	boolean	True = recibe, False = opt-out
updated_at	timestamp	Cuándo se cambió

10.3 Tabla notification_queue

Cola interna procesada por workers en background. Soporta retry automático con backoff exponencial.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
recipient_user_id	UUID	FK a users
template_id	UUID	FK a notification_templates
channel	enum	Canal a usar
payload	jsonb	Variables para el template
scheduled_for	timestamp	Cuándo enviar
status	enum	queued, processing, sent, failed, cancelled
attempts	integer	Cantidad de intentos
last_error	text	Último error si falló

10.4 Tabla notifications_sent

Log permanente de todas las notificaciones efectivamente enviadas. Para auditoría, debugging, y para evitar duplicados.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
recipient_user_id	UUID	FK a users
channel	enum	Canal usado
event_type	enum	Tipo de evento
sent_at	timestamp	Cuándo se envió
provider	string	resend, twilio, wati
provider_message_id	string	ID externo del proveedor
delivered_at	timestamp	Cuándo se confirmó entrega (vía webhook)
opened_at	timestamp	Cuándo se abrió (si trackeable)

11. Dominio: Auditoría y reportería

Capa de observabilidad del negocio. Permite responder preguntas como '¿por qué bajó la asistencia en marzo?' o '¿qué porcentaje de alumnos que empezaron el plan Trial siguen activos a los 6 meses?'

11.1 Tabla event_log

Log central de eventos de negocio (no de aplicación). Patrón de event sourcing parcial: complementa las tablas operativas, no las reemplaza.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
event_type	string	student_enrolled, payment_received, etc.
actor_user_id	UUID	Quién lo gatilló (nullable: eventos del sistema)
target_type	string	Sobre qué entidad (student_profile, course, etc.)
target_id	UUID	ID de la entidad
metadata	jsonb	Contexto del evento (estado anterior, nuevo, etc.)
occurred_at	timestamp	Cuándo ocurrió

Catálogo inicial de event_types

- Identidad: user_registered, user_logged_in, account_linked, student_profile_created.
- Comercial: subscription_started, subscription_renewed, subscription_canceled, plan_upgraded, plan_downgraded.
- Académico: student_enrolled, student_dropped, class_attended, class_missed, course_completed.
- Financiero: invoice_issued, payment_received, payment_failed, refund_processed.
- Operativo: cupo_offered_from_waitlist, teacher_replaced, class_canceled.

11.2 Estrategia de reportería por capacidad

Diferentes etapas del crecimiento de LA TROUPE requieren diferentes infraestructuras de reporte:

Etapa	Volumen	Infraestructura
MVP (≤500 alumnos)	Bajo	PostgreSQL + vistas materializadas + índices bien pensados
Crecimiento (500-2000)	Medio	Read replica de PostgreSQL para queries analíticos pesados
Multi-tenant maduro	Alto	Data warehouse separado (BigQuery/Snowflake) con ETL nocturno

Reportes obligatorios desde MVP

- Ingresos del mes por canal y método de pago.
- Deuda pendiente con aging (0-30 días, 30-60, 60+).
- Ocupación de cada class_session (cupo usado / cupo total).
- Asistencia agregada por curso, por profesor, por alumno.
- Bajas del mes con categoría de cancelación.
- Horas acumuladas por alumno (para certificación de carrera de danza).

12. Dominio: Configuración y plataforma

Este dominio implementa el principio 2.8 (business rules as data). En lugar de hardcodear políticas comerciales en el código, se almacenan en la tabla organization_settings, donde el owner las edita desde el panel admin. Las políticas pueden cambiar sin desplegar código nuevo.

12.1 Tabla organization_settings

Motor central de configuración por organización. Cada setting tiene una clave única, un valor flexible (JSON), una categoría y un flag de editabilidad. Los nombres de las claves son convenciones acordadas entre backend y frontend.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	FK a organizations
setting_key	string	Ej: 'minor_age_threshold', 'refund_policy_type'
setting_value	jsonb	Valor actual (string, number, boolean, array, object)
value_type	enum	string, number, boolean, json
category	enum	identity, finance, content, operations, communications, auth
description	text	Texto explicativo para el panel admin
is_editable	boolean	Si el admin puede cambiarlo desde la UI
updated_by	UUID	Quién lo cambió por última vez
updated_at	timestamp	Cuándo se cambió

Catálogo inicial de settings

Los settings se agrupan por categoría para facilitar la organización del panel admin:

Categoría: Identity (identidad y menores)

setting_key	value_type	Default	Descripción

minor_age_threshold	number	13	Edad desde la cual un alumno puede tener cuenta propia
guardian_required_until_age	number	18	Edad hasta la cual el tutor es responsable de pagos
allow_self_signup_minor	boolean	false	Si los menores pueden registrarse sin tutor
require_photo_consent	boolean	true	Si pedir consentimiento de fotos al inscribirse

Categoría: Finance (políticas financieras)

setting_key	value_type	Default	Descripción
default_currency_local	string	UYU	Moneda para alumnos uruguayos
default_currency_international	string	USD	Moneda para alumnos internacionales
refund_policy_type	enum	discretionary	none, credit, proportional, discretionary
enable_stripe	boolean	false	Activar Stripe para pagos internacionales
mercadopago_enabled	boolean	true	Activar MercadoPago
family_discount_enabled	boolean	true	Permitir planes familiares
teacher_as_student_discount_pct	number	50	Descuento para profesores que toman clases (0-100)
allow_credit_packs	boolean	false	Activar venta de cuponeras de clases sueltas

Categoría: Operations (operativa)

setting_key	value_type	Default	Descripción
waitlist_offer_hours	number	48	Horas que tiene el alumno para aceptar oferta de cupo
default_grace_period_online_days	number	3	Tolerancia técnica para planes digitales
default_grace_period_presencial_days	number	15	Tolerancia humana para planes presenciales
no_show_penalty_enabled	boolean	false	Si los no-show generan penalización
allow_late_attendance	boolean	true	Permitir registrar 'llegada tarde' como estado

Categoría: Auth (autenticación)

setting_key	value_type	Default	Descripción
auth_providers_enabled	array	['password','google']	Métodos de login activos
allow_apple_signin	boolean	false	Activar Apple Sign-In
require_email_verification	boolean	true	Forzar verificación de email
session_duration_days	number	30	Duración de la sesión activa

12.2 Tabla platform_subscriptions

Cuando el sistema empiece a vender licencias del software a otras academias (modelo SaaS, Fase 6), esta tabla registra qué academias están pagando a la plataforma y cuánto. Es la facturación de tu empresa hacia las academias clientes, distinta de las invoices de cada academia hacia sus alumnos.

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	Qué academia paga
platform_plan	enum	trial, starter, growth, enterprise
mrr_amount	decimal	Monthly Recurring Revenue

currency	string	Moneda del SaaS (típicamente USD)
billing_cycle	enum	monthly, yearly
status	enum	trial, active, past_due, suspended, churned
trial_ends_at	timestamp	Si está en trial
started_at	timestamp	Cuándo se sumó al SaaS
churned_at	timestamp	Si se fue, cuándo
churn_reason	string	Motivo de baja

Esta tabla se activa en Fase 6 del roadmap. Hasta entonces, no necesita crearse físicamente, pero se reserva el espacio conceptual.

13. Casos de uso críticos resueltos

13.1 Padre con dos hijos menores compra plan familiar

Juan Pérez tiene dos hijas (Sofía 8 años, Lucía 11 años) que toman clases en LA TROUPE.

- 1. Juan se registra con su email juan@gmail.com vía Google OAuth → se crea users + user_auth_providers + memberships.
- 2. Juan crea perfiles para sus hijas: 2 registros en student_profiles con account_user_id = Juan.user_id, guardian_relationship = parent.
- 3. Juan compra el 'Plan Híbrido Familiar 2 Hijos' (max_students = 2) → se crea subscriptions + 2 registros en subscription_students.
- 4. El sistema genera una invoice prorrateada (es plan presencial con calendar fijo) → Juan paga vía MercadoPago → se crea payments + se actualiza invoice.status = paid.
- 5. El sistema otorga automáticamente: 2 registros en access (uno por hija) + 2 registros en entitlements (uno por hija, location_id = NULL = todas las sedes).
- 6. Sofía y Lucía ya pueden inscribirse a clases.

13.2 Alumno se da de baja a mitad de semestre

Pedro estaba inscripto a 'Urbano Avanzado' los martes 19h en sede Montevideo. Se da de baja.

- 1. Pedro o el staff cambia class_enrollments.status = dropped, dropped_at = now().
- 2. El cupo de la class_session queda libre (capacity - active_enrollments = 1 cupo disponible).
- 3. El sistema consulta waitlist WHERE course_id AND location_id ORDER BY position.
- 4. Encuentra a María en posición 1, status = waiting → cambia su status = offered, offered_at = now(), offer_expires_at = now() + 48h.
- 5. Sistema dispara notificación: email + WhatsApp + in-app a María: 'Se liberó un cupo en Urbano Avanzado'.
- 6. Si María acepta dentro de 48h → se crea class_enrollments para ella, su status en waitlist = accepted.
- 7. Si María no responde → status = expired y se le ofrece a la siguiente.

13.3 Alumno paga factura vencida en Abitab

Carla recibió su factura mensual del 'Plan Híbrido' pero olvidó pagarla. Entró en grace_period.

- 1. Día +1 del vencimiento: subscription.status = grace_period (sigue con accesos por 15 días).
- 2. Sistema dispara notificación de aviso vía email + WhatsApp.
- 3. Día +5: Carla recibe segunda notificación. Decide pagar.

- 4. Carla genera boleta digital desde la app → se imprime/manda a su WhatsApp con código de barras.
- 5. Carla va a Abitab y paga en efectivo.
- 6. Abitab notifica a MercadoPago vía webhook → MercadoPago notifica a LA TROUPE vía webhook.
- 7. Sistema procesa: crea payments con channel = online_self_service, method = redes_cobranza, gateway = abitab.
- 8. Sistema actualiza invoice.amount_paid y, si quedó saldada, invoice.status = paid.
- 9. Sistema cambia subscription.status = active automáticamente.
- 10. Sistema dispara notificación de confirmación a Carla.

13.4 Profesor titular falta y entra sustituto

Martín (titular) avisa que no puede dictar la clase del jueves. La academia consigue a Roxana como sustituta.

- 1. Staff edita class_session_teachers: cambia el registro de Martín a status = replaced.
- 2. Staff crea nuevo registro en class_session_teachers con teacher_id = Roxana, role = substitute, status = scheduled.
- 3. Sistema dispara notificación a alumnos inscriptos: 'La clase del jueves será dictada por Roxana'.
- 4. Día de la clase: Roxana pasa lista → se crean attendance.recorded_by = Roxana.user_id.
- 5. Después de la clase: status de Roxana en class_session_teachers = taught.
- 6. Fin de mes: cálculo de teacher_payouts incluye esta clase para Roxana, no para Martín, porque la atribución viene de class_session_teachers.

13.5 Alumna intenta reservar en sede a la que no tiene acceso

Valentina contrató un plan económico (solo virtual). Intenta inscribirse a la sesión presencial en Montevideo.

- 1. Valentina hace click en 'Inscribirme' en la sesión Martes 19h Montevideo.
- 2. Sistema valida: ¿existe entitlement para Valentina con course_id = X y (location_id = Montevideo OR location_id = NULL)?
- 3. La query devuelve 0 filas: solo tiene entitlement con location_id = Online.
- 4. Sistema rechaza la inscripción y muestra mensaje: 'Tu plan actual solo te permite asistir a clases virtuales. Upgradeá a plan híbrido para asistir presencialmente.'
- 5. Sistema sugiere upgrade y deeplink al plan correspondiente.

13.6 María vincula su cuenta a una segunda academia

María es alumna de LA TROUPE desde hace un año. Academia Z, una nueva academia que entró al sistema, le ofrece tomar clases. María intenta registrarse en Academia Z.

- 1. María va al sitio de Academia Z (academiaz.com) y hace click en 'Registrarme'.
- 2. Ingresa su email maria@gmail.com.
- 3. El sistema detecta: este email ya tiene una cuenta global en la plataforma.
- 4. Sistema responde: 'Ya tenés una cuenta. ¿Querés conectar tu cuenta existente a Academia Z?'
- 5. María acepta. El sistema le pide autenticarse con su método existente (Google OAuth o password).
- 6. Una vez autenticada, se crea un membership nuevo: user_id = María, organization_id = Academia Z, system_role = student.
- 7. María es redirigida a Academia Z con sesión activa. La UI muestra un selector de organización en el header (similar a Slack).
- 8. María completa el onboarding específico de Academia Z (perfil de alumno, intereses, etc.). Se crea student_profile en Academia Z, separado del de LA TROUPE.
- 9. Sus datos en LA TROUPE permanecen privados: Academia Z nunca los ve. Lo único compartido es la identidad básica (email, nombre).

13.7 Owner cambia política de reembolsos desde el panel admin

La dirección de LA TROUPE decide cambiar la política de reembolsos: pasar de 'sin reembolso' a 'reembolso proporcional' a partir del próximo trimestre.

- 1. El Owner entra al panel admin → sección Settings → categoría Finance.
- 2. Encuentra el setting refund_policy_type con valor actual 'none'.
- 3. Cambia el valor a 'proportional'. El sistema valida y guarda en organization_settings.
- 4. La tabla event_log registra el cambio: setting_changed con metadata { setting_key, old_value, new_value, changed_by }.
- 5. A partir de ese momento, cuando un alumno cancele, el motor de cancelaciones lee organization_settings.refund_policy_type = proportional y aplica la lógica de cálculo proporcional.
- 6. Las cancelaciones anteriores no se ven afectadas (la lógica solo aplica desde el momento del cambio).
- 7. El cambio NO requiere despliegue de código. El Owner adaptó el comportamiento del sistema en 30 segundos desde la UI.

13.8 Hija cumple mayoría de edad y se independiza del perfil familiar

Lucía es alumna de LA TROUPE desde los 11 años. Su padre Juan administra su perfil. El día 14 de junio Lucía cumple 18 años y quiere gestionar sus propias reservas y pagar con su propia tarjeta. Este escenario es el ejemplo perfecto del valor del modelo Guardian-Student: la transición se resuelve como una transferencia de propiedad sin perder ningún historial.

El problema en plataformas convencionales

En la mayoría de plataformas educativas, cuando un menor crece y quiere su propia cuenta, el flujo es: borrar la cuenta infantil, crear una nueva cuenta de adulto, y perder todo el historial de clases, certificados y progreso. Es una experiencia frustrante y un fallo arquitectónico.

La solución elegante: Ownership Transfer

En LA TROUPE, esto es una simple transferencia de propiedad del student_profile. El student_profile_id de Lucía nunca cambia: solo cambia quién es su 'dueño' administrativo.

- 1. Disparador biológico: el sistema detecta que Lucía cumplió 18 años porque student_profiles.birth_date así lo indica. Se puede configurar una alerta automática para Juan o un aviso en su panel.
- 2. Lucía crea su propia cuenta de usuario con su email personal (lucia@gmail.com). Esto crea un nuevo registro en users completamente independiente del de su padre.
- 3. El sistema verifica el email de Lucía (verificación estándar para evitar suplantación).
- 4. Juan o un admin de la academia ejecuta la transferencia desde el panel: el student_profile de Lucía cambia su account_user_id (deja de apuntar a Juan y pasa a apuntar a Lucía).
- 5. El campo guardian_relationship cambia de 'parent' a 'self'.
- 6. event_log registra el cambio: ownership_transferred con metadata { student_profile_id, old_account_user_id, new_account_user_id, reason: 'majority_age' }.
- 7. Si Lucía tiene una suscripción activa que pagaba Juan, se le ofrece a Lucía hacerse cargo del próximo ciclo (con su propia tarjeta) o continuar con Juan como pagador. Esto es una decisión separada y configurable.

Lo que cada parte experimenta después de la transferencia

- Padre (Juan): al entrar a su panel 'Mi Familia' ya no ve a Lucía. Ya no recibe sus facturas ni puede reservarle clases. El sistema asume que ella ya no es su dependiente.
- Hija (Lucía): al iniciar sesión con su nuevo email ve su perfil completo, con todo su historial intacto. Sus asistencias acumuladas, sus reservas pasadas, sus certificados, sus notas médicas: todo sigue en su student_profile,

simplemente ahora ella es la dueña administrativa.

- Academia: para los profesores y staff, Lucía sigue siendo la misma alumna de siempre. Los reportes históricos no se ven afectados. La asistencia que tomó cuando tenía 12 años sigue contando para su carrera de danza.
- Sistema: las relaciones uno-a-muchos (attendance, enrollments, access, entitlements) están atadas al student_profile_id, que es inmutable. Por eso ningún historial se pierde al transferir el ownership.

Por qué esto es la prueba del valor del modelo Guardian-Student

Este caso de uso demuestra el principio de la sección 2.5: identidad de cuenta vs identidad de alumno son entidades distintas. La cuenta de login (users) es la 'llave de control'; el perfil de alumno (student_profiles) es la 'identidad operativa'. Transferir la llave no destruye la identidad. Esta separación arquitectónica, decidida en Fase 1, paga dividendos durante toda la vida del producto.

14. Stack tecnológico recomendado

Stack productivo definido para LA TROUPE: optimizado para costo cero o casi-cero en MVP, escalable a miles de alumnos sin reescritura, y con buena documentación. La decisión central es usar Supabase como BaaS (Backend-as-a-Service), lo que reduce drásticamente el código de infraestructura que el equipo tiene que escribir y mantener.

Disclaimer importante sobre los costos en USD

Los costos en dólares que aparecen a lo largo de esta sección son referenciales para escenarios de contratación de devs externos a tarifa de mercado (USD 25-40/h para senior Latam). NO aplican al escenario actual de LA TROUPE, donde el desarrollo se hace internamente sin contratar devs externos. En ese caso el costo en USD del desarrollo es cero, pero el costo real es el tiempo de calendario invertido.

Para el equipo interno conviene mirar la columna de complejidad relativa (1-10) que aparece en las tablas detalladas más abajo. Esa métrica es independiente del modelo económico y refleja la dificultad técnica real de cada feature, lo que ayuda a priorizar y a calcular tiempo de calendario aproximado.

14.1 Stack productivo en una tabla

Costos de infraestructura mensual. Estos son costos de servicios externos y aplican igual a cualquier escenario (contratación de devs externos o desarrollo interno).

Componente	Proveedor	Función	Costo MVP
Frontend	Vercel	Next.js + edge network global	USD 0
BaaS (Backend + DB + Auth)	Supabase	PostgreSQL, Auth, Edge Functions, Storage, Realtime	USD 0 (Free tier)
Video streaming	Bunny.net	Almacenamiento + streaming con HLS Encryption	USD 1-5
Reproductor	Vidstack (free)	Player React-first con extensibilidad	USD 0
Pagos Latam	MercadoPago	Tarjetas locales, Abitab, RedPagos	USD 0 (comisión)
Pagos globales	Stripe	Multi-currency + métodos locales internacionales	USD 0 (comisión)
Email transaccional	Resend	Notificaciones y comunicaciones	USD 0 (3000 envíos/mes)
Error tracking	Sentry	Monitoreo de errores en producción	USD 0 (5000 errores/mes)
WhatsApp (Fase 3)	Twilio o Wati	Notificaciones proactivas	USD 50-150

Costo total estimado de infraestructura del MVP: USD 5-10/mes. A 500 alumnos activos: USD 50-100/mes. A 2000 alumnos activos: USD 200-400/mes. Esta economía es crítica para una academia en etapa inicial. Estos costos son de servicios externos y se incurren independientemente de quién haga el desarrollo.

14.2 Por qué Supabase es la decisión central

Supabase reemplaza varios componentes que tradicionalmente requieren servicios separados o desarrollo a medida:

- PostgreSQL gestionada con Row-Level Security nativo: exactamente lo que necesitamos para multi-tenant.
- Auth integrada con email/password y OAuth (Google, Apple, etc.) con account linking soportado.
- Storage para archivos (thumbnails, certificados, documentos).
- Edge Functions: serverless TypeScript para webhooks (MercadoPago, Stripe), tareas programadas y APIs custom.
- Realtime: actualización en vivo para notificaciones in-app y dashboards (vía WebSocket).
- pg_cron: tareas programadas (cobros automáticos, vencimientos, recordatorios) sin servicios externos.
- Tier gratuito generoso: 500MB de DB, 1GB de storage, 50K usuarios activos mensuales. Suficiente para MVP completo.

14.3 Frontend (Next.js sobre Vercel)

Componente	Recomendación	Justificación
Framework	Next.js 14+ con App Router	SSR para SEO de cursos públicos, RSC para performance
UI components	shadcn/ui + Tailwind CSS	Componentes copiables, sin lock-in, diseño moderno
State management	TanStack Query + Zustand	Server state via Supabase + client state minimalista
Forms	React Hook Form + Zod	Validación type-safe, integrable con Supabase schemas
Mobile (Fase 4+)	React Native con Expo	Reuso de lógica de Next.js, deploy simple

14.4 Video y media (Bunny.net Stream)

Componente	Recomendación	Justificación
Hosting de video	Bunny.net Stream	5-10x más barato que Vimeo, CDN en Sudamérica
Protección anti-piratería	HLS Encryption + Token Auth	Sweet spot: detiene piratería casual
Reproductor	Bunny embed o Plyr custom	Bunny embed alcanza para MVP
Almacenamiento auxiliar	Supabase Storage	Documentos, certificados, avatares

Estimación de costo Bunny: 50 alumnos viendo 10GB cada uno = 500GB de bandwidth = ~USD 5/mes. A 200 alumnos × 15GB = 3000GB = ~USD 30/mes. La economía sigue siendo excelente comparada con alternativas.

14.5 Pagos: Smart Payment Routing

LA TROUPE tiene dos audiencias distintas: alumnos uruguayos/latinoamericanos (mercado primario) y alumnos internacionales que compran cursos on-demand (mercado secundario). Cada audiencia tiene métodos de pago preferidos distintos. La estrategia es smart payment routing: enrutar a cada alumno al gateway óptimo según su país.

Componente	Recomendación	Justificación
Gateway primario (MVP)	MercadoPago	Mejor cobertura en Uruguay y Latam, incluye Abitab/RedPagos
Gateway internacional (Fase 2)	Stripe	Multi-currency, métodos locales por país (SEPA, OXXO, Apple Pay)
Gateway evaluación (Fase 3+)	dLocal	Solo si los volúmenes justifican fees mínimos mensuales
POS físico	MercadoPago Point	Reportería unificada con online, hardware barato
Routing automático	Por país del alumno	Detección vía IP/perfil + opción manual

Lógica de enrutamiento por país

País del alumno	Gateway primario	Gateway secundario
Uruguay	MercadoPago (tarjetas locales, Abitab, RedPagos)	Stripe (fallback)
Argentina, Brasil, Chile, México, Colombia	MercadoPago (penetración fuerte)	Stripe (fallback)
España, EE.UU., resto del mundo	Stripe (multi-currency + métodos locales)	—

Implicancias arquitectónicas del multi-gateway

- El modelo de datos ya soporta múltiples gateways: el campo payments.gateway acepta cualquier proveedor. No hay cambios en el schema.
- Cada gateway requiere su propio webhook handler (en Supabase Edge Functions).
- Patrón Event Bus interno: los webhooks externos se traducen a eventos canónicos (PaymentApproved, PaymentFailed, SubscriptionRenewed) que el resto del sistema consume sin saber de qué gateway provienen.
- Las suscripciones recurrentes tienen API distinta en cada proveedor. La capa de abstracción interna oculta esta diferencia.
- Webhook signing: cada gateway firma sus webhooks de forma distinta. Es crítico validar firmas para evitar webhooks falsificados.

14.6 Comunicación

Canal	Proveedor	Fase	Notas
Email transaccional	Resend	MVP	3000 envíos gratis/mes, escala a SES si crece mucho
In-app	Supabase Realtime	MVP	WebSocket integrado, sin servicio adicional
WhatsApp Business	Twilio o Wati	Fase 3	Templates pre-aprobadas obligatorias
SMS	Twilio	Fase 3	Solo para alertas críticas (costo acumulativo)

14.7 DevOps y observabilidad

Componente	Recomendación	Costo
CI/CD	GitHub Actions + Vercel auto-deploy	USD 0
Error tracking	Sentry	USD 0 (tier gratis hasta 5K errores/mes)
Logs	Supabase Logs + Vercel Logs	USD 0 (incluido)
Uptime monitoring	UptimeRobot	USD 0 (50 monitores gratis)
Migraciones de DB	Supabase Migrations	USD 0 (incluido)
Secretos	Vercel Environment Variables + Supabase Secrets	USD 0 (incluido)

14.8 Reproductor de video: Vidstack + features avanzadas

El reproductor es el corazón de la experiencia de aprendizaje en una plataforma de danza. Las plataformas de referencia (Steezy, Rebaila, Dance Masterclass) compiten por la calidad de su reproductor. La decisión: usar Vidstack como base por su API React-first, integración nativa con HLS de Bunny, y customización vía CSS variables sin pelearse con styling legacy.

Features del reproductor por fase de implementación

La columna 'complejidad' es relativa (1-10), independiente del modelo económico de desarrollo. La columna 'costo dev' aplica solo a escenarios de contratación externa.

Feature	Fase	Complejidad	Costo dev externo	Riesgo
Setup Vidstack + integración Bunny	1	3/10	USD 720	Bajo
Speed control (preservesPitch)	1	2/10	USD 480	Bajo
Capítulos con tabla lesson_chapters	1	4/10	USD 960	Bajo
Modo Espejo robusto	1	5/10	USD 1200	Medio (edge cases)
Loop A-B con pre-buffer	2	8/10	USD 1680	Alto
Cámara local (webcam PIP)	4	6/10	USD 1440	Medio
Cambiar Vistas opción D (PIP editado)	5	5/10	USD 960	Comercial primero

Total Fase 1 del reproductor: complejidad acumulada 14/40, costo dev externo referencial USD 4560. En desarrollo interno este se traduce en aproximadamente 3-4 semanas de calendario para un dev solo trabajando part-time, con consultas asistidas para resolver edge cases.

Decisiones técnicas concretas del reproductor

- Speed control debe usar preservesPitch = true para evitar audio agudo/grave en velocidades distintas a 1x. Detectar Safari iOS antiguo y mostrar aviso si no soporta.
- Modo Espejo aplica transform: scaleX(-1) solo al elemento video, NO al contenedor del player (los controles se invertirían). Subtítulos se renderizan en capa separada sin transform.
- Loop A-B requiere pre-buffer del segmento del loop y fade audio de 50ms antes/después del seek para evitar 'click' audible.
- Cámara local debe aplicar transform: scaleX(-1) automáticamente a la webcam (la persona espera verse como en un espejo).
- Cambiar Vistas: NO incluir en MVP. Es decisión comercial primero porque cambia cómo graba la academia (segunda cámara, doble edición). Cuando se evalúe, recomendar opción D (picture-in-picture editado en post-producción) por mejor balance costo/UX.

15. Roadmap de 6 fases

Roadmap incremental que prioriza retorno de inversión y mitigación de riesgos. Cada fase entrega valor concreto al negocio. Una decisión clave de v3.2 es separar la captura de datos (siempre en Fase 1) de las features visibles que consumen esos datos (pueden ser fase posterior), aplicando el principio 2.10.

Fase 1 — MVP: Venta on-demand + panel admin + captura desde día uno

Objetivo: que LA TROUPE pueda vender su primer curso digital, cobrarlo online, que el owner pueda configurar el sistema sin intervención de desarrollo, y que TODOS los datos relevantes empiecen a capturarse desde el primer alumno. 12-14 semanas con 2 devs (calendario más amplio si el desarrollo es interno y part-time).

- Identidad: users (con is_platform_admin), user_auth_providers (email + Google OAuth), memberships, organizations, locations, student_profiles, staff_profile (minimal).
- Contenido: lessons, courses (con allowed_modes), course_lessons, lesson_assets (Bunny.net integrado), lesson_chapters.
- Captura de datos desde día uno (principio 2.10): lesson_progress activa, lesson_video_views activa, event_log activo. Sin UI consumidora todavía, pero los eventos se capturan.
- Comercial básico: purchases, payments, invoices simples, MercadoPago integrado.
- Autorización: access (sin entitlements aún).
- Configuración: organization_settings con catálogo inicial de settings + panel admin para editarlos.

- Panel admin completo: CRUD de planes, cursos, lecciones, sedes, capítulos; vista de alumnos; configuración de organización.
- Reproductor de video con Vidstack: Speed control, Capítulos, Modo Espejo (las 3 features básicas que diferencian a una plataforma de danza de un Vimeo genérico).
- Frontend público: home, catálogo de cursos, checkout, mi-cuenta, reproductor de video.
- Reportes básicos: ingresos del mes, alumnos activos, ventas por curso.

Fase 2 — Ingresos recurrentes y mercado internacional

Objetivo: convertir compras puntuales en suscripciones mensuales y abrir el producto a alumnos fuera de Uruguay. 6-8 semanas.

- plans, subscriptions, subscription_students.
- Generación dinámica de invoices con grace_period_days por plan.
- Lógica de suspensión automática cuando vence grace_period.
- Prorrateo para planes con calendario fijo.
- Webhook handlers de MercadoPago en Supabase Edge Functions.
- Integración de Stripe como gateway internacional (multi-currency).
- Webhook handlers de Stripe traducidos al Event Bus interno.
- Smart payment routing: detección de país del alumno + selección de gateway óptimo.
- Capa de abstracción para suscripciones que oculte las diferencias entre Stripe Subscriptions y MercadoPago Recurring.

Fase 3 — El mundo híbrido

Objetivo: soportar las clases presenciales y virtuales en vivo. Esta es la fase más grande. 10-14 semanas.

- class_sessions (con flexibilidad de location_id por sesión), masterclasses, class_session_teachers.
- class_enrollments con validación contra entitlements y waitlist.
- attendance: módulo de check-in para profesor y staff.
- waitlist con notificaciones automáticas.
- Módulo de notificaciones completo: templates, preferences, queue, sent log.
- Calendario de clases en frontend (alumno y profesor).
- Integración con Zoom para sesiones virtuales (creación y unión a meetings).
- Panel admin extendido: programación de sesiones individuales con location específica por sesión, asignación de profesores.

Fase 4 — Retención, mobile y analítica (UI consumiendo datos capturados)

Objetivo: maximizar retención y dar herramientas de decisión al Owner. La data está capturada desde Fase 1 (lesson_progress, lesson_video_views, event_log); en esta fase se construye la UI que la consume. 8-12 semanas.

- UI 'Continuá donde quedaste': consume lesson_progress (capturado desde Fase 1) para mostrar última posición y porcentaje completado por lección.
- Dashboard del Owner con analítica avanzada: ingresos, asistencia, ocupación, retención, churn. Consume event_log y lesson_video_views.
- Reportes de comportamiento: 'qué partes de las lecciones repiten más los alumnos', 'qué velocidad de reproducción usan'. Consume lesson_video_views.
- App móvil (React Native con Expo): clave para clases presenciales y notificaciones push.
- Apple Sign-In activado para la app móvil iOS.

- Cámara local en el reproductor (feature exclusiva de la app móvil donde tiene mejor sentido).
- WhatsApp Business integrado vía Twilio o Wati.
- Read replica de PostgreSQL para queries analíticos (Supabase plan Pro).

Fase 5 — Pagos a profesores y RRHH completo

Objetivo: profesionalizar la liquidación a profesores y agregar el módulo completo de RRHH del staff. La tabla staff_profile ya existe desde Fase 1 con datos básicos; en esta fase se enriquece con campos avanzados y se construye la UI completa. 6-8 semanas.

- teacher_compensation_rules con vigencia temporal.
- Cálculo automático de teacher_payouts sumando attendance + reglas vigentes.
- Aprobación y registro de pagos a profesores.
- Reportes individuales por profesor: clases dictadas, alumnos enseñados, monto a cobrar.
- Extensión de staff_profile con campos avanzados: compensación, vacaciones, evaluaciones, comisiones de venta.
- UI completa de gestión de RRHH: el dato histórico desde Fase 1 ya está disponible para reportes.
- Panel admin: gestión de reglas de compensación por profesor y por staff.

Fase 6 — SaaS multi-tenant + Platform Admin

Objetivo: incorporar la primera academia externa al sistema y comenzar a vender el software como servicio. Activable cuando el negocio lo justifique. 10-12 semanas.

- platform_subscriptions: tabla de facturación de tu empresa hacia las academias.
- Platform Admin Panel: vista cross-tenant exclusiva para is_platform_admin = true.
- Onboarding de nuevas organizaciones: wizard de configuración inicial automatizado.
- Flujo de vinculación de cuenta existente a nueva academia (caso de uso 13.6).
- Selector de organización en frontend (UX tipo Slack workspace switcher).
- Personalización por organización: colores, logo, dominio propio (custom domain).
- Soporte multi-currency y multi-timezone real entre academias.
- Métricas SaaS para Platform Owner: MRR, churn de academias, conversión de trials.

16. Decisiones tomadas vs delegadas al panel admin

16.1 Decisiones tomadas

Cada una de estas decisiones tiene una justificación documentada y un impacto concreto en el modelo de datos.

Área	Decisión	Impacto
Inscripciones	Enrollments semestrales + attendance por sesión	Resuelve cupo persistente y reportes de horas
Lista de espera	Waitlist FIFO con oferta de 48h	Reduce fricción operativa
Profesores	class_session_teachers con role y status	Soporta sustituciones auditables
No-show	Sin penalización, tracking obligatorio	Mantiene experiencia humana
Modalidad de inscripción	Campo enrollment_mode (semester/weekly/drop_in)	Soporta drop-in futuro sin refactor
Hosting de video	Bunny.net Stream	Costo ~5x menor que Vimeo
Anti-piratería	HLS Encryption + Token Auth	Bloquea piratería casual

Modelo de assets	Tabla lesson_assets provider-agnostic	Migrable sin reescribir
Contenido gratuito	is_preview por asset + is_free por lesson	Doble estrategia de marketing
Login	Email + password + Google OAuth	Reduce fricción 30-50%
Account linking	Auto-link silencioso con verificación	Patrón Stripe/Notion
Modelo de menores	Guardian-Student con student_profiles	Resuelve familias y adultos
Multi-tenancy	By design, single-tenant by deployment	Listo para escalar sin refactor
Contenido por org	Privado por organización	Modelo SaaS estándar
Aislamiento de datos	PostgreSQL Row-Level Security	Red de seguridad arquitectónica
Suscripciones familiares	Por plan vía max_students	Flexibilidad sin complejidad
Taxonomía de pagos	Channel × method × gateway	Reportería precisa
Procesador de pagos	MercadoPago en MVP, Stripe en Fase 2	Smart routing por país del alumno
Configurabilidad	organization_settings + panel admin	Business rules as data, no hardcode
Modalidad por sesión	courses.allowed_modes + class_sessions.location_id	Lecciones híbridas dentro de un mismo curso
Identidad federada	users global + memberships múltiples	Soporta usuarios cross-academy
Niveles jerárquicos	Platform Owner vs Organization Owner	Vía users.is_platform_admin
Stack tecnológico	Vercel + Supabase + Bunny + MP/Stripe + Resend	Costo MVP: USD 5-10/mes
Captura de datos temprana	Tablas de captura activas desde Fase 1	Principio 2.10: lo que no se captura hoy no se inventa mañana
Reproductor de video	Vidstack como base + features por fase	Speed/Capítulos/Mirror Fase 1, Loop A-B Fase 2, Cámara Fase 4
Modelo de capítulos	Tabla lesson_chapters (no JSON dentro de lessons)	Permite editar individualmente y agregar metadata futura
staff_profile minimal en Fase 1	Tabla creada desde día uno con campos básicos	RRHH histórico disponible cuando se construya UI en Fase 5

16.2 Decisiones delegadas al panel admin

La filosofía v3.0 establece que las decisiones comerciales no se hardcodean en el código sino que viven en organization_settings, editables desde el panel admin. Esto refleja el principio 2.8 (business rules as data). El owner de la academia puede adaptar estas políticas según el mercado, sin requerir cambios de software.

Tema	Setting / Mecanismo	Quién decide y cuándo
Cuponeras	settings.allow_credit_packs + plans con type='cuponera'	Owner cuando quiera ofrecerlas
Profesor como alumno	settings.teacher_as_student_discount_pct	Owner según política comercial vigente
Pago a profesores	teacher_compensation_rules por profesor	Admin al contratar/renegociar con cada profesor
Apple Sign-In	settings.allow_apple_signin	Admin cuando lance app móvil
Política de menores	settings.minor_age_threshold + guardian_required_until_age	Owner según asesoría legal
Política comercial familiar	Crear plans con max_students y precio combinado	Owner cuando defina catálogo de planes

Capacidad virtual	class_sessions.capacity por sesión	Admin al programar cada sesión
Política de cancelación	settings.refund_policy_type	Owner según estrategia comercial
Audiencia internacional	settings.enable_stripe + analítica de país	Owner al evaluar datos de Fase 1
Moneda	plans.currency por plan	Owner al crear cada plan
Modalidad de cada lección	class_sessions.location_id por sesión específica	Admin al programar sesiones (presencial/virtual flexible por lección)

Esta sección reemplaza la antigua lista de 'decisiones pendientes' de v2. La filosofía cambió: en lugar de pedirle a la academia que decida políticas estáticas, le entregamos un sistema configurable donde puede adaptar las políticas según necesidad.

Próximos pasos sugeridos

- Validar el modelo de datos final con un revisor técnico independiente antes de empezar a codear.
- Diseñar wireframes de las 10-12 pantallas críticas (catálogo, checkout, mi-cuenta, calendario, panel admin).
- Definir el catálogo inicial de planes que tendrá LA TROUPE: nombres, precios, max_students, grace_period_days.
- Setup de cuentas: Vercel, Supabase, MercadoPago, Bunny.net, Resend, Sentry (todas en modo sandbox/free tier).
- Definir el equipo de desarrollo: 2 devs full-stack con experiencia en Next.js + Supabase es el mínimo viable.
- Confirmar valores iniciales para los settings de organization_settings (defaults razonables, ajustables después).
- Crear el repositorio del proyecto con CI/CD básico (GitHub Actions + Vercel auto-deploy).

17. Glosario

Términos técnicos y de negocio usados en este documento.

Término	Definición
Access	Derecho a consumir contenido digital. Tabla del vault.
Account linking	Conectar múltiples métodos de login a una misma cuenta.
Allowed modes	Array que declara qué modalidades son posibles para un curso (presencial, virtual o ambas). La modalidad concreta de cada lección se decide a nivel sesión.
Attendance	Registro real de asistencia de un alumno a una class_session.
Audit log	Registro inmutable de eventos para trazabilidad. Ver event_log.
Calendar billing	Ciclo de facturación con vencimiento fijo (día 1 del mes).
Captura temprana de datos	Principio que establece que las tablas de captura de eventos deben crearse desde el día uno del proyecto, aunque la UI consumidora se construya en fases posteriores.
Channel (pago)	Dónde se inicia un pago: online_self_service o pos_in_store.
Class session	Evento específico (curso × sede × fecha).
Configuration over hardcoding	Principio que establece que las reglas de negocio cambiantes deben vivir en datos editables, no en código.
Drip	Liberación progresiva de contenido (lección N se desbloquea en día N).
Drop-in	Modalidad de inscripción de clase suelta sin compromiso semestral.
Entitlement	Derecho a ocupar cupo en una sede. Tabla del vault.
Enrollment	Inscripción del alumno a un curso (semestral, weekly o drop-in).

Event sourcing parcial	Patrón: registrar eventos de negocio en log central, complementando tablas operativas.
Federated Identity	Modelo donde un usuario tiene una sola cuenta global pero puede pertenecer a múltiples organizaciones vía membresías.
Grace period	Período de tolerancia tras vencer una factura antes de suspender accesos.
Guardian-Student	Modelo donde una cuenta administra múltiples perfiles de alumno.
Híbrido	Modalidad que combina presencial y virtual.
HLS Encryption	Protección de video por cifrado de segmentos.
Independencia de perfil	Proceso por el cual un student_profile administrado por un tutor pasa a ser auto-gestionado (típicamente al cumplir la mayoría de edad). Implementado vía Ownership Transfer.
Invoice	Obligación de pago. Distinta del payment (ingreso real).
Masterclass	Evento premium de pago único, generalmente con profesor invitado.
Method (pago)	Instrumento usado para pagar: cash, card, mercadopago, etc.
Multi-tenant	Arquitectura que sirve a múltiples organizaciones con la misma base de código.
Organization Owner	Dueño de una academia individual (ej: dueño de LA TROUPE). Control total dentro de su organización.
Organization Settings	Tabla central donde viven las reglas comerciales editables desde el panel admin.
Ownership Transfer	Patrón arquitectónico para reasignar el control administrativo de un student_profile sin perder su historial. Cambia account_user_id pero conserva student_profile_id. Usado típicamente cuando un menor cumple la mayoría de edad.
Platform Owner	Dueño del software (tu empresa). Gestiona las academias clientes del SaaS. Distinto del Organization Owner.
POS	Point of Sale: terminal de cobro físico en sede.
Prorateo	Cálculo proporcional de la primera factura cuando se inscribe a mitad de período.
Provider-agnostic	Diseño que no depende de un proveedor específico; permite migración.
RBAC	Role-Based Access Control. Sistema de permisos basado en roles.
Redes de cobranza	Abitab, RedPagos: pago en efectivo en agencias físicas con código de barras.
Rolling billing	Ciclo de facturación cada 30 días desde la compra.
Row-Level Security	Feature de PostgreSQL que filtra automáticamente filas por organización.
SaaS	Software as a Service. Modelo donde se vende el software como servicio.
Smart payment routing	Enrutar al alumno al gateway de pago óptimo según su país, automáticamente.
Staff role	Sub-rol operativo (cashier, receptionist) para segregación de funciones.
Student profile	Identidad de alumno, distinta de la cuenta de login.
Subscription	Suscripción recurrente a un plan.
Token authentication	Protección de URLs de video con tokens temporales firmados.
Vault	Conjunto de tablas que gestionan derechos: access + entitlements.
Vidstack	Reproductor de video moderno hecho específicamente para React/Vue/Svelte. Base elegida para el reproductor de LA TROUPE por su API React-first y soporte HLS nativo.
Waitlist	Lista de espera FIFO para cupos liberados.
Webhook	Llamada HTTP que un proveedor externo hace al sistema cuando ocurre un evento (pago aprobado, etc.)
YAGNI	You Aren't Gonna Need It. Principio clásico que dice 'no construyas lo que no necesitás hoy'. En LA TROUPE se aplica selectivamente: SÍ para features visibles, NO para captura de datos (ver principio 2.10).

Fin del documento maestro de arquitectura

LA TROUPE — versión 3.2 — abril 2026